W5 High Performance Python (2) Matrix and Vector Computation

Department of Smart Mobility Engineering
Inwook, Shim
인하대학교
INHA UNIVERSITY

Today

- Goal
  - Understanding Vector computation
  - Right usage of Python matrix/vector.

- Content
  - Diffusion equation
  - `list` implementation
  - Memory fragmentation
  - `numpy` implementation

Vector and Matrix

- Wherever you are, whatever you do, someday, you will face a problem that must be solved through vector/matrix computation.
  - Vector/matrix calculation is essential part to how a computer work and how it tries to speed up.
  - Computer only knows simple operation repeatedly.

New example for vector computation

Introduction to diffusion in fluids

- Ink in water w/o no stirring, not warm: we assume that diffusion is the main process mixing the two liquids.

$$\frac{\partial}{\partial t}u(x,t) = D \cdot \frac{\partial^2}{\partial x^2}u(x,t)$$

유체확산 / 1차 편미분 방정식
u(확산하는 유체의 양),

- u (0~1): vector for the quantities of diffusing.
  u = 0 vector, pure water state, and u = 1 vector, pure ink state.
  This is 1D example. 2D/3D vector indicates area/volume.

- D : diffusion coefficient.
  Large D indicates easily diffusion.
  For simplification, we set D = 1.

If you want to know more about diffusion in fluids, please read Ch.7 of 'Numerical methods for complex systems S.V. Gurevich' and Ch.17 of 'Numerical Recipes' 3rd edition by William Press

Approximate diffusion equation

$$\frac{\partial}{\partial t}u(x,t) = D \cdot \frac{\partial^2}{\partial x^2}u(x,t)$$

식 변형 : u(x,t)가 주어졌을 때, u(x, t+dt)를 계산할 수 있도록
- dt >> time resolution
- dx >> data resolution e.g., 데이터 해상도

- Approximate diffusion equation with discrete volumes and times.

  >> We use *Euler's method.*

$$\frac{\partial}{\partial t}u(x,t) \approx \frac{u(x,t+dt) - u(x,t)}{dt}$$

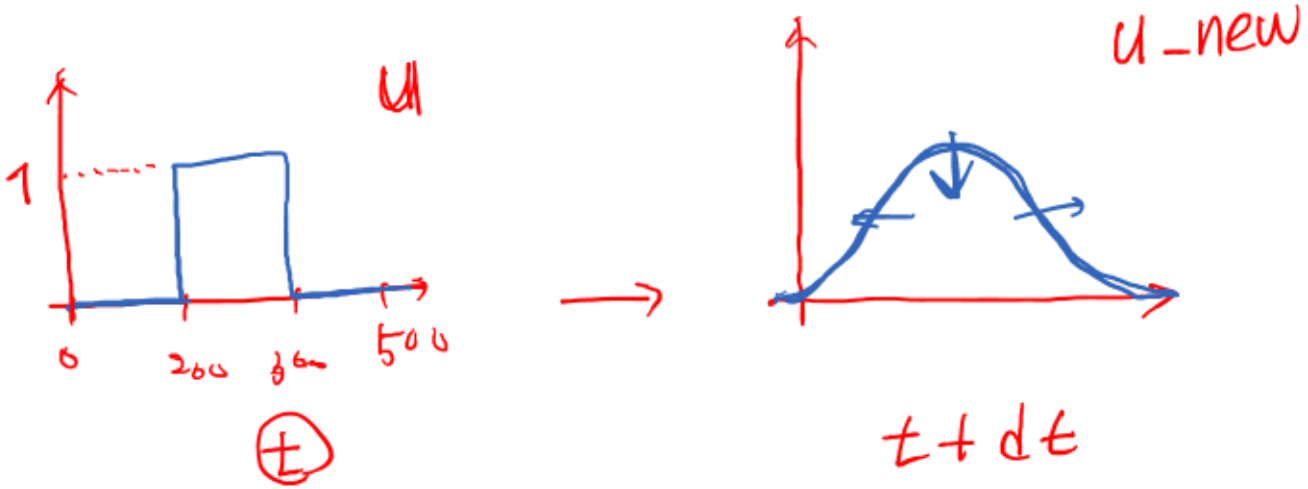
, where dt is a constant number representing time step (time resolution.) The smaller resolution, the more accuracy.

$$u(x,t+dt) = u(x,t) + dt * D * \frac{u(x+dx,t) - 2u(x,t) + u(x-dx,t)}{dx^2}$$

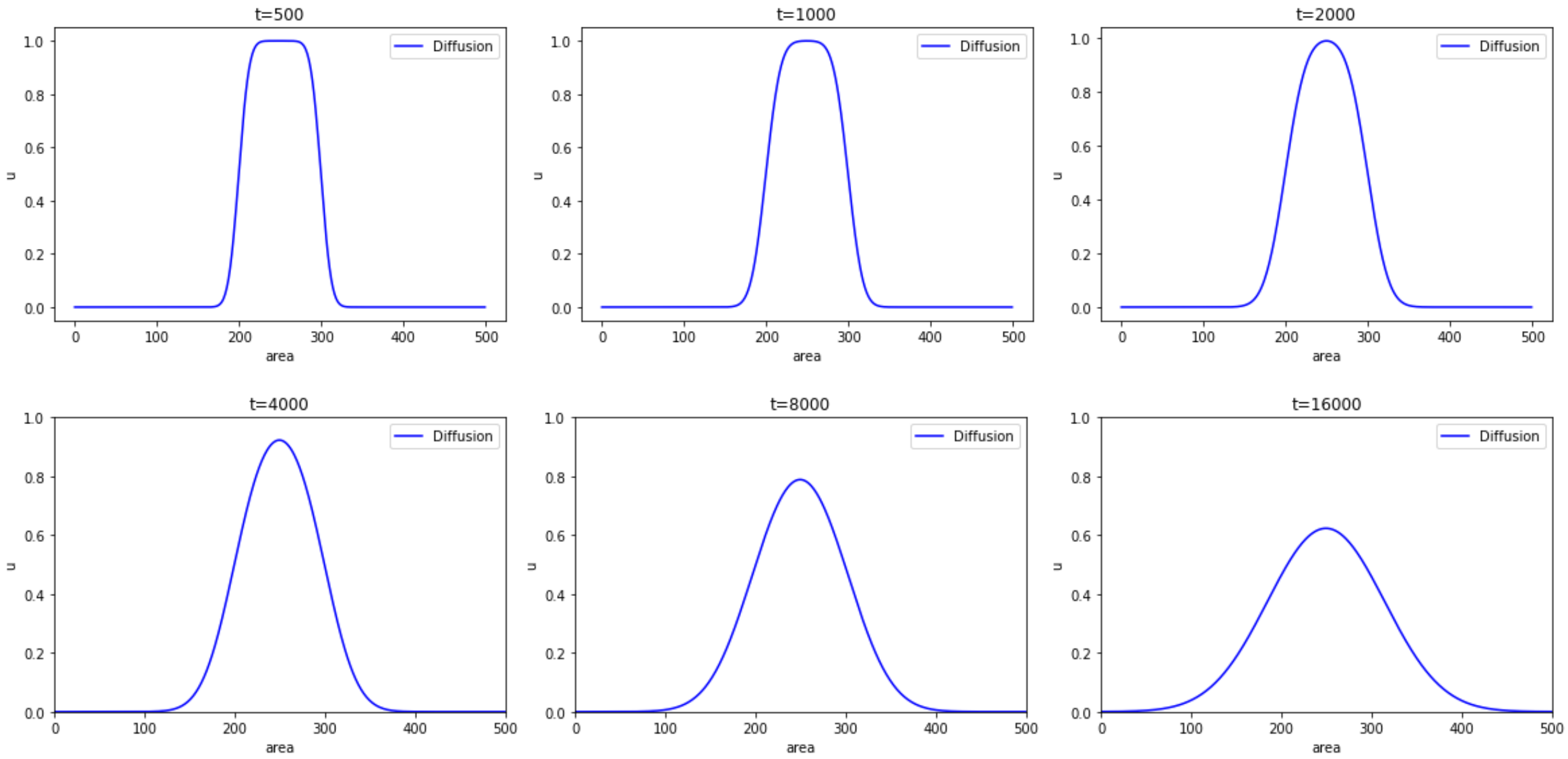
Initial state: $u(x,0)$, we set dt = 1 and D = 1.

Code implementation 1D example

```
N = 500
water = [0] * N
ink = [1] * 100
u = 0
D = 1
t = 0
dt = 0.1

u_init = water
u_init[200:300] = ink
u = u_init

while True:
  u_new = [0] * N
  for i in range(N):
    u_new[i] = u[i] + D * dt * (u[ (i + 1) % N ] + u[(i-1)%N] - 2 * u[i])
  u = u_new
  t += 1
  if t == 500:
    break;
```



Code implementation 1D example



Introduction to 2D diffusion in fluids

- Calculating diffusion will be operated over 2D matrix.

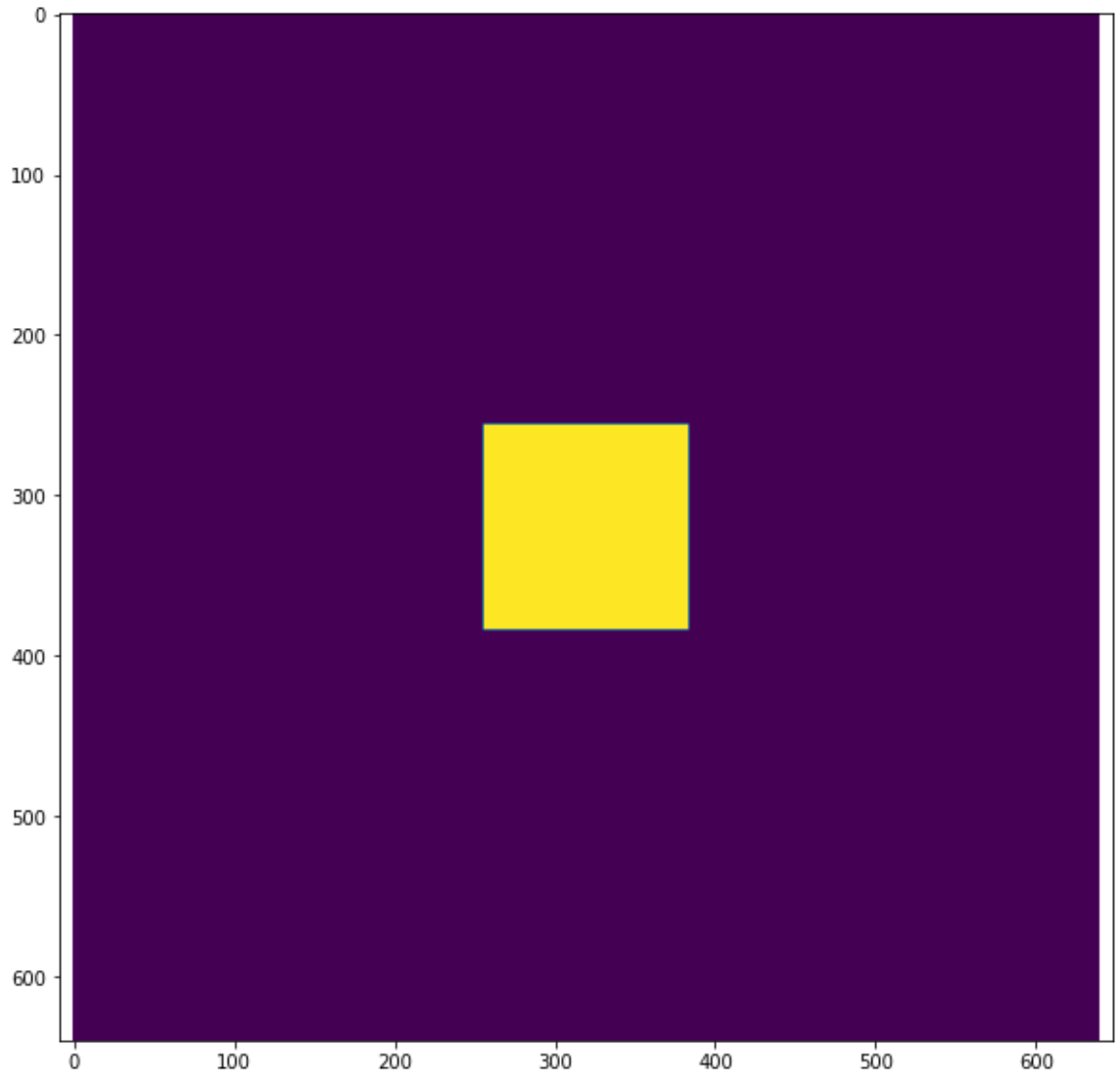
$$\frac{\partial}{\partial t}u(x,y,t) = D\left(\frac{\partial^2}{\partial x^2}u(x,y,t) + \frac{\partial^2}{\partial y^2}u(x,y,t)\right)$$

```
for i in range(N):
  u_new[i] = u[i] + D * dt * (u[ (i + 1) % N ] + u[(i-1)%N] - 2 * u[i])
```

```
for i in range(N):
  for i in range(M):
    u_new[i][j] = u[i][j] + D * dt * (
                                       (u[(i + 1) % N][j] + u[(i - 1) % N][j] - 2 * u[i][j]) +
                                       (u[i][(j + 1) % N] + u[i][(j - 1) % N] - 2 * u[i][j])
                                      )
```

Toy example in 2D space

Initial state

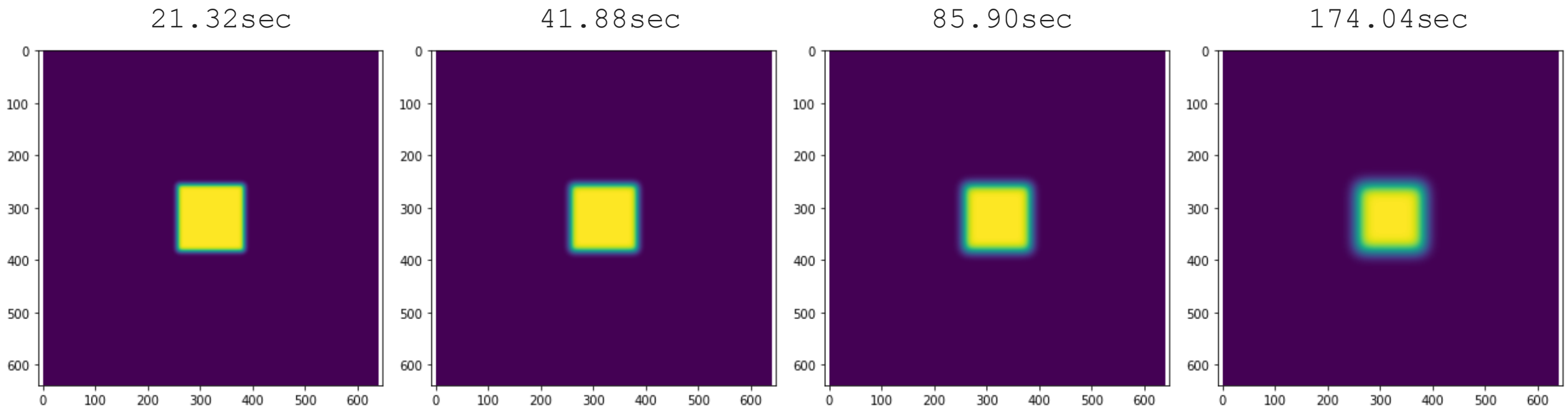



```
data_size = (640,640)
u = [[0.0] * ymax for x in range(xmax)]

#intialization
ink_low = int(data_size[0] * 0.4)
ink_high = int(data_size[0] * 0.6)
for i in range(ink_low, ink_high):
  for j in range(ink_low, ink_high):
    u[i][j] = 0.005
```

Introduction to 2D diffusion in fluids

- Naive implementation

```
for i in range(xmax):
  for j in range(ymax):
    dxx = (u_in[(i + 1) % xmax][j] + u_in[(i - 1) % xmax][j] - 2.0 * u_in[i][j])
    dyy = (u_in[i][(j + 1) % ymax] + u_in[i][(j - 1) % ymax] -2.0 * u_in[i][j])
    u_new[i][j] = u_in[i][j] + D * dt * (dxx + dyy)
```



t=100 t=200 t=400 t=800

Code example

```
data_size = (640,640)
def diffusion(u_in, dt, D=1.0):
  xmax, ymax = data_size
  u_new = [[0.0] * ymax for x in range(xmax)]
  for i in range(xmax):
    for j in range(ymax):
      dxx = (u_in[(i + 1) % xmax][j] +
             u_in[(i - 1) % xmax][j] –
             2.0 * u_in[i][j])
      dyy = (u_in[i][(j + 1) % ymax] +
             u_in[i][(j - 1) % ymax] –
             2.0 * u_in[i][j])
      u_new[i][j] = u_in[i][j] + D * dt * (dxx + dyy)
  return u_new
```

```
def dropInk(max_iter):
  xmax, ymax = data_size
  u = [[0.0] * ymax for x in range(xmax)]

  #intialization
  ink_low = int(data_size[0] * 0.4)
  ink_high = int(data_size[0] * 0.6)
  for i in range(ink_low, ink_high):
    for j in range(ink_low, ink_high):
      u[i][j] = 0.005
  u_init = u

  start = time.time()
  for i in range(max_iter):
    u = diffusion(u, 0.1)
  end = time.time()
  return end-start, u_init, u
```

line_profiler: 2D diffusion in fluids

```
root@docker-desktop:/workspace/hpp/w6# kernprof -lv diffusion.py
Time:183.63630151748657sec
Wrote profile results to diffusion.py.lprof
Timer unit: 1e-06 s


Total time: 108.976 s
File: diffusion.py
Function: diffusion at line 6


Line #      Hits         Time  Per Hit   % Time  Line Contents
==============================================================
     6                                           @profile
     7                                           def diffusion(u_in, dt, D=1.0):
     8       100       265.0      2.6      0.0    xmax, ymax = data_size
     9       100     52434.0    524.3      0.0    u_new =  [[0.0] * ymax for x in range(xmax)]
    10     64100     23838.0      0.4      0.0    for i in range(xmax):
    11  41024000  14638117.0      0.4     13.4      for j in range(ymax):
    12  40960000  14365541.0      0.4     13.2        dxx = (
    13  40960000  23363566.0      0.6     21.4            u_in[(i + 1) % xmax][j] + u_in[(i - 1) % xmax][j] - 2.0 * u_in[i][j]
    14                                                 )
    15  40960000  14227616.0      0.3     13.1        dyy = (
    16  40960000  22537565.0      0.6     20.7            u_in[i][(j + 1) % ymax] + u_in[i][(j - 1) % ymax] -2.0 * u_in[i][j]
    17                                                 )
    18  40960000  19767035.0      0.5     18.1        u_new[i][j] = u_in[i][j] + D * dt * (dxx + dyy)
    19       100       158.0      1.6      0.0    return u_new
```

line_profiler: 2D diffusion in fluids

```
root@docker-desktop:/workspace/hpp/w6# kernprof -lv diffusion.py
Time:183.63630151748657sec
Wrote profile results to diffusion.py.lprof
Timer unit: 1e-06 s

Total time: 108.976 s
File: diffusion.py
Function: diffusion at line 6

Line #      Hits         Time  Per Hit   % Time  Line Contents
==============================================================
     6                                           @profile
     7                                           def diffusion(u_in, dt, D=1.0):
     8       100       265.0      2.6      0.0    xmax, ymax = data_size
     9       100     52434.0    524.3      0.0    u_new =  [[0.0] * ymax for x in range(xmax)]
    10     64100     23838.0      0.4      0.0    for i in range(xmax):
    11  41024000  14638117.0      0.4     13.4        for j in range(ymax):
    12  40960000  14365541.0      0.4     13.2            dxx = (
    13  40960000  23363566.0      0.6     21.4                u_in[(i + 1) % xmax][j] + u_in[(i - 1) % xmax][j] - 2.0 * u_in[i][j]
    14                                                    )
    15  40960000  14227616.0      0.3     13.1            dyy = (
    16  40960000  22537565.0      0.6     20.7                u_in[i][(j + 1) % ymax] + u_in[i][(j - 1) % ymax] -2.0 * u_in[i][j]
    17                                                    )
    18  40960000  19767035.0      0.5     18.1            u_new[i][j] = u_in[i][j] + D * dt * (dxx + dyy)
    19       100       158.0      1.6      0.0    return u_new
```

- 640+1 : loop 종료 조건을 만나기 위해 한번 더 실행
- Line 9: Per Hit, 0.000524 초, 한 번당 속도가 가장 느림
  전체 시간은 0.052434초
  루프가 덜 호출되니까 %Time은 매우 작음.
  (데이터 크기 줄이고, 함수 호출이 늘어나면, 대부분의 시간을 차지하게 될 거라고 예측 가능.)

Retrieved from the global namespace

Call 64,100 times
- 64,100 = (xmax=640 + 1) * iter=100

This line informs us that the function is profiled 100 times.

Code example memory reduction

Remove memory allocation code line

```
data_size = (640,640)
def diffusion(u_in, u_out, dt, D=1.0):
  xmax, ymax = data_size
  for i in range(xmax):
    for j in range(ymax):
      dxx = (u_in[(i + 1) % xmax][j] +
             u_in[(i - 1) % xmax][j] –
             2.0 * u_in[i][j])
      dyy = (u_in[i][(j + 1) % ymax] +
             u_in[i][(j - 1) % ymax] –
             2.0 * u_in[i][j])
      u_out[i][j] = u_in[i][j] + D * dt * (dxx + dyy)
```

```
def dropInk(max_iter):
  xmax, ymax = data_size
  u = [[0.0] * ymax for x in range(xmax)]
  u_new = [[0.0] * ymax for x in range(xmax)]

  #intialization
  ink_low = int(data_size[0] * 0.4)
  ink_high = int(data_size[0] * 0.6)
  for i in range(ink_low, ink_high):
    for j in range(ink_low, ink_high):
      u[i][j] = 0.005
  u_init = copy.deepcopy(u)
  start = time.time()
  for i in range(max_iter):
    diffusion(u, u_new, 0.1)
    u, u_new = u_new, u
  end = time.time()
  return end-start, u_init, u
```

line_profiler: 2D diffusion in fluids

```
root@docker-desktop:/workspace/hpp/w6# kernprof -lv diffusion_mem_reduction.py
Time:144.30414271354675sec
Wrote profile results to diffusion_mem_reduction.py.lprof
Timer unit: 1e-06 s
```

108 s →

```
Total time: 93.3088 s
File: diffusion_mem_reduction.py
Function: diffusion at line 7

Line #      Hits         Time  Per Hit   % Time  Line Contents
==============================================================
     7                                           @profile
     8                                           def diffusion(u_in, u_out, dt, D=1.0):
     9       100       186.0      1.9      0.0    xmax, ymax = data_size
    10     64100     19030.0      0.3      0.0    for i in range(xmax):
    11  41024000  12133880.0      0.3     13.0        for j in range(ymax):
    12  40960000  11714139.0      0.3     12.6            dxx = (
    13  40960000  20544186.0      0.5     22.0                u_in[(i + 1) % xmax][j] + u_in[(i - 1) % xmax][j] - 2.0 * u_in[i][j]
    14                                                  )
    15  40960000  11704080.0      0.3     12.5            dyy = (
    16  40960000  20015451.0      0.5     21.5                u_in[i][(j + 1) % ymax] + u_in[i][(j - 1) % ymax] -2.0 * u_in[i][j]
    17                                                  )
    18  40960000  17177827.0      0.4     18.4            u_out[i][j] = u_in[i][j] + D * dt * (dxx + dyy)
```

- 불필요한 제거된 코드 결과
- **실제 프로파일링에서 나타난 시간보다 시간 감소 효과가 더 큼**.
- 메모리 요청을 OS에 요청할 때, 상당한 오버헤드
- 코드 일관성을 유지하면서 메모리 할당을 관리
- 그리고 가능하다면 이미 할당된 메모리를 재사용하는 방향을 고려

← mem. allocation

Memory fragmentation

- There is no native vector/matrix operation in Python.
- Python list store memory address (like pointer in C.) `int *a;`
  - Actual data is located at some other place.
  - Unnecessary overhead (jump operation, fragmented memory transfer) makes slow our program.
- Python bytecode is **not** optimized for vectorization.

Performance Counter

-naïve implementation for 2D diffusion-

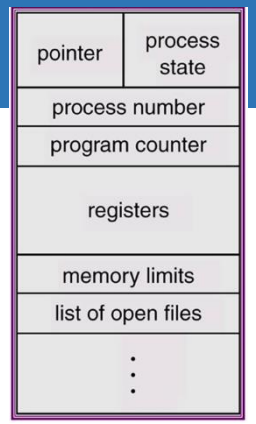
Performance counter for 2D diffusion

```
$ perf stat -e cycles,instructions,cache-references,cache-misses,branches,branch-misses,task-clock,faults,minor-faults
,cs,migrations\
python3 diffusion_mem_reduction.py
```

```
Performance counter stats for 'python3 diffusion_mem_reduction.py':

  50,690,202,303      cycles                    #    3.094 GHz
 221,140,436,515      instructions              #    4.36  insn per cycle
      55,451,931      cache-references          #    3.385 M/sec
       7,731,831      cache-misses              #   13.943 % of all cache refs
  44,852,604,330      branches                  #    2.738 G/sec
       1,026,183      branch-misses             #    0.00% of all branches
      16,381.88 msec task-clock                 #    1.000 CPUs utilized
          10,309      faults                    #  629.293 /sec
          10,309      minor-faults              #  629.293 /sec
              85      cs                        #    5.189 /sec
               1      migrations                #    0.061 /sec

    16.384311852 seconds time elapsed

    16.346807000 seconds user
     0.035997000 seconds sys
```

Analyzing perf

```
     50,690,202,303      cycles                    #    3.094 GHz
    221,140,436,515      instructions              #    4.36  insn  per cycle
         55,451,931      cache-references          #    3.385 M/sec
          7,731,831      cache-misses              #   13.943 % of all cache refs
     44,852,604,330      branches                  #    2.738 G/sec
          1,026,183      branch-misses             #    0.00% of all branches
          16,381.88 msec task-clock                #    1.000 CPUs utilized
             10,309      faults                    #  629.293 /sec
             10,309      minor-faults              #  629.293 /sec
                 85      cs                        #    5.189 /sec
                  1      migrations                #    0.061 /sec
```



Process Control Block (PCB)

task-clock	How many clock cycle is used for a task. (소비한 총 클럭수 ex) cpu2개로 1초가 걸렸다면, task-clock은 2)	
instructions	How many CPU instructions is executed.	💡|instruction – cycles| indicates how well the code is vectorized.
cycles	How many cycles take to run all of instructions.	
cs (context-switching)	How long the program is paused, because of kernel operation(I/O), other apps, move execution to another CPU core.	💡context-switching is extremely time-consuming task, but we don't have to o much control over when cs happens.
migrations	How many time the program stopped and resumed different CPU.	
faults (page-fault)[1]	When the memory is first used, OS throws a minor page fault interrupt. (a major page fault also happens when the program request data from device (disk, network, etc.)	
cache-reference	Increase, whenever the program refers data from cache.	
cache-misses	Increase, whenever fetch data from RAM.	

[1] https://scoutapm.com/blog/understanding-page-faults-and-memory-swap-in-outs-when-should-you-worry

Analyzing perf

```
     50,690,202,303      cycles                    #    3.094 GHz
    221,140,436,515      instructions              #    4.36  insn per cycle
         55,451,931      cache-references          #    3.385 M/sec
          7,731,831      cache-misses              #   13.943 % of all cache refs
     44,852,604,330      branches                  #    2.738 G/sec
          1,026,183      branch-misses             #    0.00% of all branches
          16,381.88 msec task-clock                #    1.000 CPUs utilized
             10,309      faults                    #  629.293 /sec
             10,309      minor-faults              #  629.293 /sec
                 85      cs                        #    5.189 /sec
                  1      migrations                #    0.061 /sec
```

branch	The number of the execution flow changed.	💡if…then, CPU estimates directions the branches and preload next instructions in advance.
branch-misses	When branch prediction is incorrect, branch-misses occurs	💡sorted list v.s. unsorted list

- Perf can track lots of metrics to analyzing your program, and it <u>depends on what CPU do you use</u>.

Analyzing perf

32 core / RAM 128GB from system

```
Performance counter stats for 'python3 diffusion_mem_reduction.py':

  50,690,202,303      cycles                    #    3.094 GHz
 221,140,436,515      instructions              #    4.36  insn per cycle
      55,451,931      cache-references          #    3.385 M/sec
       7,731,831      cache-misses              #   13.943 % of all cache refs
  44,852,604,330      branches                  #    2.738 G/sec
       1,026,183      branch-misses             #    0.00% of all branches
      16,381.88 msec task-clock                #    1.000 CPUs utilized
          10,309      faults                    #  629.293 /sec
          10,309      minor-faults              #  629.293 /sec
              85      cs                        #    5.189 /sec
               1      migrations                #    0.061 /sec

  16.384311852 seconds time elapsed

  16.346807000 seconds user
   0.035997000 seconds sys
```

13% = cache-misses / cache-references
cache-references 55,451,931
cache-misses 7,731,831

CPU performs an average 4.36 instruction per cycle.
(pipelining boost, out-of-order execution, hyperthreading)

Memory fragmentation increases the number of memory transfers from RAM to CPU because of bus can delivers contiguous chunks of memory.
`list` stores contiguous memory address, but the actual values are scattered in memory. That is CPU can not copy all at once, if the memory is highly fragmented.

Revisiting numpy

- numpy provides many things What We Want.
  - store data in contiguous chunks of memory.
  - support vectorized operations

  → We can calculate many arithmetic operations w/o loop.

추천: https://www.labri.fr/perso/nrougier/from-python-to-numpy/

Performance comparison btw. `list` and numpy

```python
def norm_square_list(vector):
  norm=0
  for v in vector:
    norm += v * v
  return norm
```

```
>>> vector = list(range(1_000_000))
>>> %timeit norm_square_list(vector)
>>> 97.2 ms ± 820 µs per loop (mean ± std. dev. of
7 runs, 10 loops each)
```

```python
def norm_square_list_comprehension(vector):
  return sum([v * v for v in vector])
```

```
>>> %timeit norm_square_list_comprehension(vector)
>>> 113 ms ± 2.87 ms per loop (mean ± std. dev. of
7 runs, 10 loops each)
```

```python
def norm_square_array(vector):
  norm=0
  for v in vector:
    norm += v * v
  return norm
```

```
>>> vector_array = array('l', range(1_000_000))
>>> %timeit norm_square_array(vector_array)
>>> 96.7 ms ± 1.73 ms per loop (mean ± std. dev. of
7 runs, 10 loops each)
```

```python
def norm_square_numpy(vector):
  return numpy.sum(vector * vector)
```

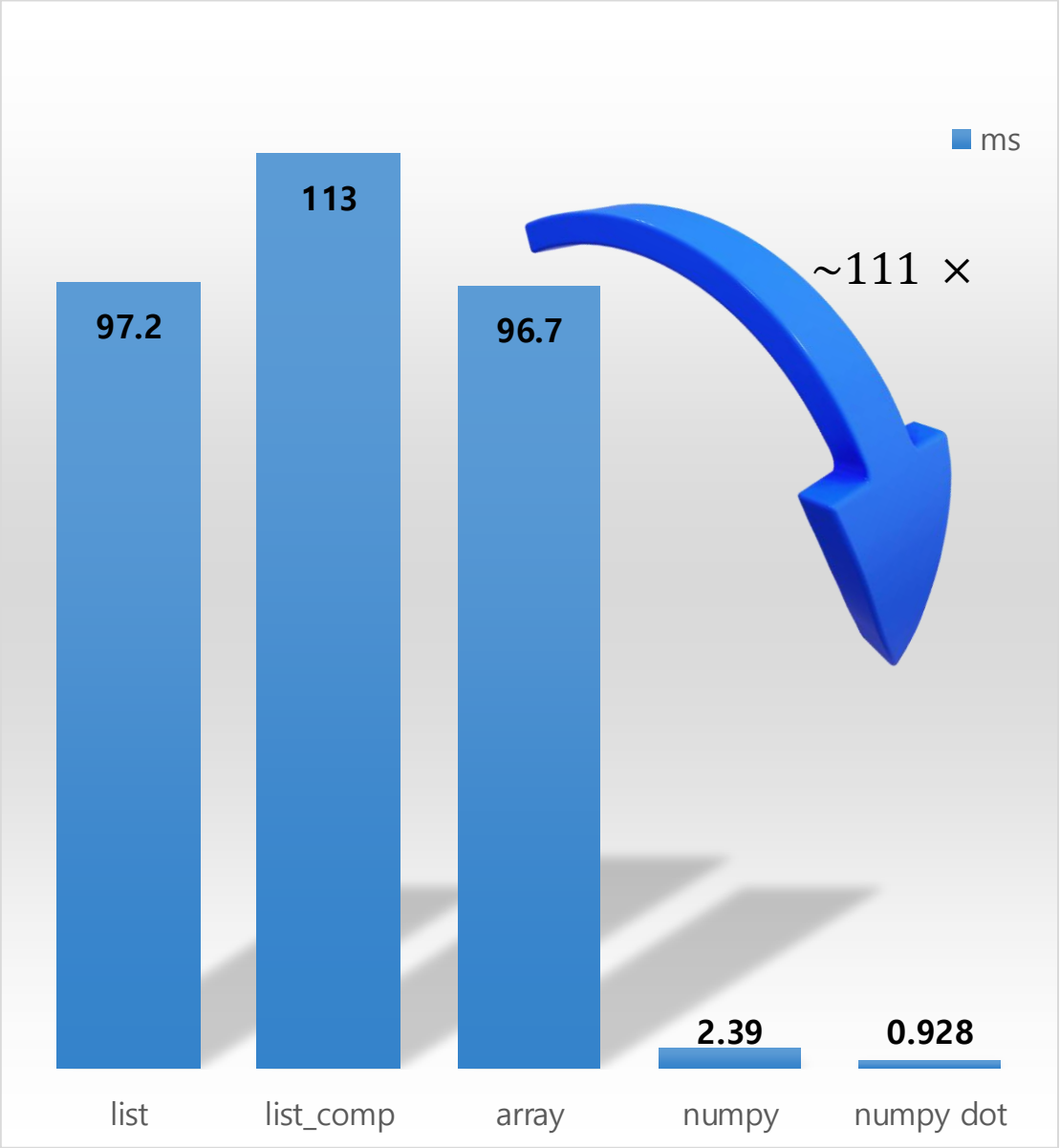
```
>>> vector_np = numpy.arange(1_000_000)
>>> %timeit norm_square_numpy(vector_np)
>>> 2.39 ms ± 84.1 µs per loop (mean ± std. dev. of
7 runs, 100 loops each)
```

```python
def norm_square_numpy_dot(vector):
  return numpy.dot(vector, vector)
```

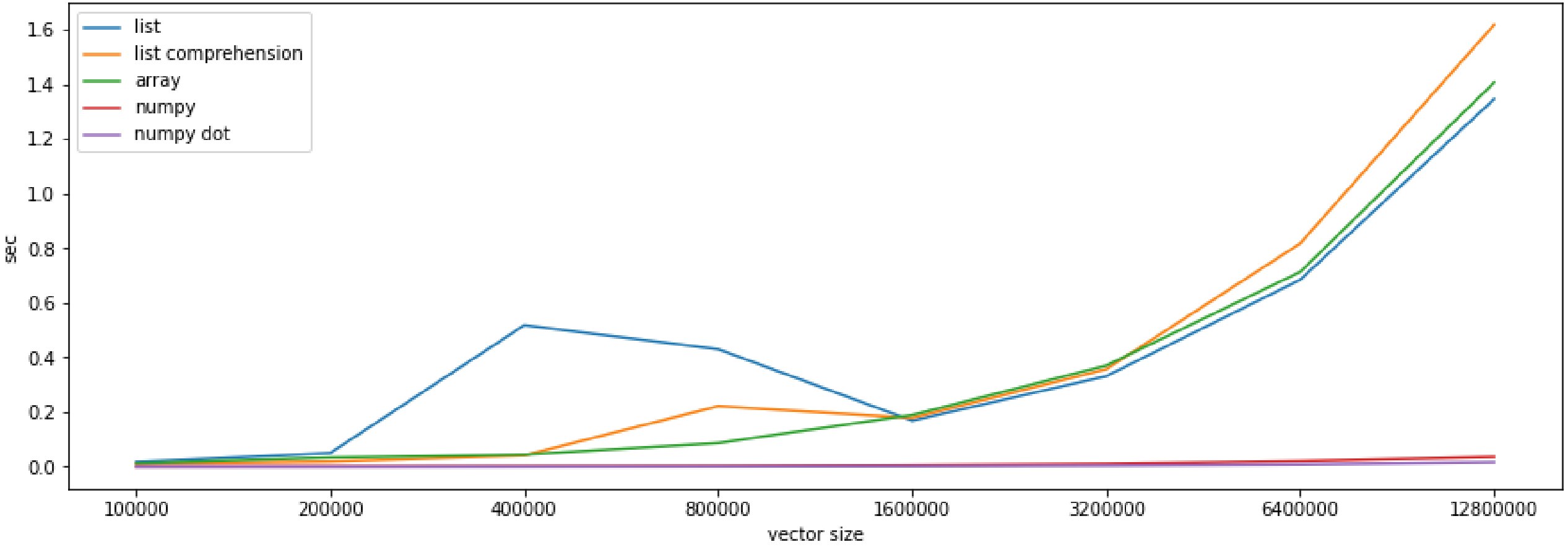
```
>>> %timeit norm_square_numpy_dot(vector_np)
>>> 928 µs ± 9.89 µs per loop (mean ± std. dev. of
7 runs, 1000 loops each)
```

Performance comparison btw. list and numpy

```python
def norm_square_list(vector):
  norm=0
  for v in vector:
    norm += v * v
  return norm

def norm_square_list_comprehension(vector):
  return sum([v * v for v in vector])

def norm_square_array(vector):
  norm=0
  for v in vector:
    norm += v * v
  return norm

def norm_square_numpy(vector):
  return numpy.sum(vector * vector)


def norm_square_numpy_dot(vector):
  return numpy.dot(vector, vector)
```



Performance comparison according to vector size



numpy

- Built-in native C code has a drastic speedup.
- Special purpose `numpy` is better than generalized list structure when we handle numerical computations.
- `numpy` gives us memory locality and vectorized operations.
- `numpy` has very optimized C code that has been made specifically to take advantage of any vectorization the CPU has enabled.

* Doing `vector * vector`, there is an implied loop that `numpy` will take care of.

Performance Counter

-numpy implementation for 2D diffusion-

Code example numpy

Numpy roll
([1,2,3,4], 1) >> [4,1,2,3]

```
data_size = (640,640)
def diffusion_op(u_in):
  return (roll(u_in, +1, 0) +
          roll(u_in, -1, 0) +
          roll(u_in, +1, +1) +
          roll(u_in, -1, +1) -
          4 * u_in)

def diffusion(u_in, dt, D=1):
  return u_in + dt * D * diffusion_op(u_in)
```

numpy.roll

numpy.roll(*a, shift, axis=None*) [source]

Roll array elements along a given axis.

Elements that roll beyond the last position are re-introduced at the first.

Parameters: a : *array_like*
Input array.

shift : *int or tuple of ints*
The number of places by which elements are shifted. If a tuple, then *axis* must be a tuple of the same size, and each of the given axes is shifted by the corresponding number. If an int while *axis* is a tuple of ints, then the same value is used for all given axes.

axis : *int or tuple of ints, optional*
Axis or axes along which elements are shifted. By default, the array is flattened before shifting, after which the original shape is restored.

Returns: res : *ndarray*
Output array, with the same shape as *a*.

```
def dropInk(max_iter):
  u = zeros(data_size)

  #intialization
  ink_low = int(data_size[0] * 0.4)
  ink_high = int(data_size[0] * 0.6)
  u[ink_low:ink_high, ink_low:ink_high] = 0.005
  start = time.time()
  for i in range(max_iter):
    u = diffusion(u, 0.1)
  end = time.time()
  return end-start, u
```

Performance counter for 2D diffusion with numpy

```
$ perf stat -e cycles,instructions,cache-references,cache-misses,branches,branch-misses,task-clock,faults,minor-faults
,cs,migrations\
python3 diffusion_numpy.py
```

```
 Performance counter stats for 'python3 diffusion_numpy.py':

    1,503,108,953      cycles                    #    2.954 GHz
    1,845,386,364      instructions              #    1.23  insn per cycle
      109,471,905      cache-references          #  215.117 M/sec
        3,534,199      cache-misses              #    3.228 % of all cache refs
      332,427,236      branches                  #  653.233 M/sec
        3,591,203      branch-misses             #    1.08% of all branches
          508.90 msec task-clock                 #    0.999 CPUs utilized
          85,035      faults                     #  167.097 K/sec
          85,035      minor-faults               #  167.097 K/sec
               0      cs                         #    0.000 /sec
               0      migrations                 #    0.000 /sec

      0.509608907 seconds time elapsed

      0.417395000 seconds user
      0.092308000 seconds sys
```

```
Performance counter stats for 'python3 diffusion_numpy.py':

   1,503,108,953      cycles                    #    2.954 GHz
   1,845,386,364      instructions              #    1.23  insn per cycle
     109,471,905      cache-references          #  215.117 M/sec
       3,534,199      cache-misses              #    3.228 % of all cache refs
     332,427,236      branches                  #  653.233 M/sec
       3,591,203      branch-misses             #    1.08% of all branches
         508.90 msec task-clock                 #    0.999 CPUs utilized
          85,035      faults                    #  167.097 K/sec
          85,035      minor-faults              #  167.097 K/sec
               0      cs                        #    0.000 /sec
               0      migrations                #    0.000 /sec

   0.509608907 seconds time elapsed

   0.417395000 seconds user
   0.092308000 seconds sys
```

Same result with reducing instruction per cycle.
cache-misses 13% → 3%
Zero context-switching
? Increase page-faults.
Extremely reduced processing time 16 sec → 0.5 sec

```
Performance counter stats for 'python3 diffusion_mem_reduction.py':

  50,690,202,303      cycles                    #    3.094 GHz
 221,140,436,515      instructions              #    4.36  insn per cycle
      55,451,931      cache-references          #    3.385 M/sec
       7,731,831      cache-misses              #   13.943 % of all cache refs
  44,852,604,330      branches                  #    2.738 G/sec
       1,026,183      branch-misses             #    0.00% of all branches
      16,381.88 msec task-clock                 #    1.000 CPUs utilized
          10,309      faults                    #  629.293 /sec
          10,309      minor-faults              #  629.293 /sec
              85      cs                        #    5.189 /sec
               1      migrations                #    0.061 /sec

   16.384311852 seconds time elapsed

   16.346807000 seconds user
   0.035997000 seconds sys
```

Memory Allocations and In-Place Operations

- Memory allocation is quite bigger burden than cache-misses.
  - Requesting to the OS for an available chunk of data and memory is more overhead than simply filling a cache.
- Simple Trick : In-Place operations (+=, *=, and so on.)

```
import numpy as np

array1 = np.random.random((10,10))
array2 = np.random.random((10,10))

array1 += array2

array1 = array1 + array2
```

Memory Allocations and In-Place Operations

- Memory allocation is quite bigger burden than cache-misses.
  - Requesting to the OS for an available chunk of data and memory is more overhead than simply filling a cache.
- Simple Trick : In-Place operations (+=, *=, and so on.)

In place operation: 입력 중 하나가 위치한 메모리에 결과값 다시 저장

```
import numpy as np

array1 = np.random.random((10,10))
array2 = np.random.random((10,10))
print(id(array1))
array1 += array2
print(id(array1))
array1 = array1 + array2
print(id(array1))
```

```
>>> 140043921465136
>>> 140043921465136
>>> 1400439214628 32
```

Performance of In-Place Operations

```
%%timeit
array1, array2 = np.random.random((2, 1000, 1000))
array1 = array1 + array2
>>> 11.2 ms ± 222 µs per loop (mean ± std. dev. of 7 runs, 100 loops each)
```

```
%%timeit
array1, array2 = np.random.random((2, 1000, 1000))
array1 += array2
>>> 10.7 ms ± 136 µs per loop (mean ± std. dev. of 7 runs, 100 loops each)
```

5% improving

- It is important to note that this effect happens only when <u>the array sizes are bigger than the CPU cache</u>!

```
%%timeit
array1, array2 = np.random.random((2, 5, 5))
array1 = array1 + array2
>>> 2.38 µs ± 49.5 ns per loop (mean ± std. dev. of 7 runs, 100000 loops each)
```

```
%%timeit
array1, array2 = np.random.random((2, 5, 5))
array1 += array2
>>> 2.75 µs ± 49.9 ns per loop (mean ± std. dev. of 7 runs, 100000 loops each)
```

Performance decreasing

Performance Counter

-In-place implementation for 2D diffusion-

Code example numpy with In-Place operator

```
data_size = (640,640)
def diffusion_op(u_in, u_out):
  np.copyto(u_out, u_in)
  u_out *= -4
  u_out += roll(u_in, +1, 0)
  u_out += roll(u_in, -1, 0)
  u_out += roll(u_in, +1, +1)
  u_out += roll(u_in, -1, +1)

def diffusion(u_in, u_out, dt, D=1):
  diffusion_op(u_in, u_out)
  u_out *= D * dt
  u_out += u_in
```

```
def dropInk(max_iter):
  u = zeros(data_size)
  u_new = zeros(data_size)

  ink_low = int(data_size[0] * 0.4)
  ink_high = int(data_size[0] * 0.6)
  u[ink_low:ink_high, ink_low:ink_high] = 0.005
  start = time.time()
  for i in range(max_iter):
    diffusion(u, u_new, 0.1)
    u, u_new = u_new, u
  end = time.time()
  return end-start, u
```

Performance counter for 2D diffusion In-place operators

```
$ perf stat -e cycles,instructions,cache-references,cache-misses,branches,branch-misses,task-clock,faults,minor-faults
,cs,migrations\
python3 diffusion_inplace.py
```

```
 Performance counter stats for 'python3 diffusion_inplace.py':

     1,289,663,038      cycles                    #    2.932 GHz
     1,554,574,423      instructions              #    1.21  insn per cycle
       104,852,495      cache-references          #  238.381 M/sec
        12,237,265      cache-misses              #   11.671 % of all cache refs
       274,143,409      branches                  #  623.261 M/sec
         3,485,377      branch-misses             #    1.27% of all branches
            439.85 msec task-clock                #    0.998 CPUs utilized
             8,807      faults                    #   20.023 K/sec
             8,807      minor-faults              #   20.023 K/sec
                 1      cs                        #    2.273 /sec
                 0      migrations                #    0.000 /sec

       0.440654486 seconds time elapsed

       0.400665000 seconds user
       0.040066000 seconds sys
```

```
Performance counter stats for 'python3 diffusion_inplace.py':

   1,289,663,038      cycles                    #    2.932 GHz
   1,554,574,423      instructions              #    1.21  insn per cycle
     104,852,495      cache-references          #  238.381 M/sec
      12,237,265      cache-misses              #   11.671 % of all cache refs
     274,143,409      branches                  #  623.261 M/sec
       3,485,377      branch-misses             #    1.27% of all branches
         439.85 msec task-clock                 #    0.998 CPUs utilized
           8,807      faults                    #   20.023 K/sec
           8,807      minor-faults              #   20.023 K/sec
               1      cs                        #    2.273 /sec
               0      migrations                #    0.000 /sec

   0.440654486 seconds time elapsed

   0.400665000 seconds user
   0.040066000 seconds sys
```

Page-fault 85,000 → 8,800

```
Performance counter stats for 'python3 diffusion_numpy.py':

   1,503,108,953      cycles                    #    2.954 GHz
   1,845,386,364      instructions              #    1.23  insn per cycle
     109,471,905      cache-references          #  215.117 M/sec
       3,534,199      cache-misses              #    3.228 % of all cache refs
     332,427,236      branches                  #  653.233 M/sec
       3,591,203      branch-misses             #    1.08% of all branches
         508.90 msec task-clock                 #    0.999 CPUs utilized
          85,035      faults                    #  167.097 K/sec
          85,035      minor-faults              #  167.097 K/sec
               0      cs                        #    0.000 /sec
               0      migrations                #    0.000 /sec

   0.509608907 seconds time elapsed

   0.417395000 seconds user
   0.092308000 seconds sys
```

- Cache-miss가 늘어났음에도 불구하고, 마이너 페이지 폴트 수가 줄어 성능향상
- 페이지 폴트는 프로그램이 새로 할당된 공간에 처음 접근할 때 발생, (그전에는 OS에서 메모리만 던져주고 실제 메모리에 접근 하지 않음)
- 커널은 프로그램 일시중지, 메모리 공간 확인 후 배정
- 프로그램을 정지하고 프로세스가 외부로 나가서 자원을 할당받고 돌아옴

What else?

- Reduce the burden of CPU using `numpy`.
- Performance profiling based code-optimization is conducted.
- Review again

What else?

```
Line #    Hits         Time  Per Hit   % Time  Line Contents
==============================================================
    16                                           @profile
    17                                           def dropInk(max_iter):
    18         1          8.0      8.0      0.0    u = zeros(data_size)
    19         1          4.0      4.0      0.0    u_new = zeros(data_size)
    20
    21         1          2.0      2.0      0.0    ink_low = int(data_size[0] * 0.4)
    22         1          1.0      1.0      0.0    ink_high = int(data_size[0] * 0.6)
    23         1        194.0    194.0      0.1    u[ink_low:ink_high, ink_low:ink_high] = 0.005
    24
    25       101         64.0      0.6      0.0    for i in range(max_iter):
    26       100     330325.0   3303.2     99.9        diffusion(u, u_new, 0.1)
    27       100        134.0      1.3      0.0        u, u_new = u_new, u
    28         1          0.0      0.0      0.0    return u
```

```
Line #    Hits         Time  Per Hit   % Time  Line Contents
==============================================================
    12                                           @profile
    13                                           def diffusion(u_in, u_out, dt, D=1):
    14       100     368709.0   3687.1     91.8    diffusion_op(u_in, u_out)
    15       100      12163.0    121.6      3.0    u_out *= D * dt
    16       100      20758.0    207.6      5.2    u_out += u_in
```

What else?

```
Total time: 0.367856 s
File: diffusion_inplace.py
Function: diffusion_op at line 4

Line #      Hits         Time  Per Hit   % Time  Line Contents
==============================================================
     4                                           @profile
     5                                           def diffusion_op(u_in, u_out):
     6       100      38411.0    384.1     10.4    np.copyto(u_out, u_in)
     7       100      34373.0    343.7      9.3    u_out *= -4
     8       100      93081.0    930.8     25.3    u_out += roll(u_in, +1, 0)
     9       100      69296.0    693.0     18.8    u_out += roll(u_in, -1, 0)
    10       100      72909.0    729.1     19.8    u_out += roll(u_in, +1, +1)
    11       100      59786.0    597.9     16.3    u_out += roll(u_in, -1, +1)
```

- numpy.roll allocates a new vector in memory.
- numpy.roll is general purpose function which means there are lots of codes for many special cases we don’t care/

Define new function for special case roll function.

```
def custom_roll(rollee, shift, axis, out):
  if shift==1 and axis==0:
    out[1:, :] += rollee[:-1, :]
    out[0, :] += rollee[-1, :]
  elif shift == -1 and axis == 0:
    out[:-1, :] += rollee[1:, :]
    out[-1, :] += rollee[0, :]
  elif shift == 1 and axis == 1:
    out[:, 1:] += rollee[:, :-1]
    out[:, 0] += rollee[:, -1]
  elif shift == -1 and axis == 1:
    out[:, :-1] += rollee[:, 1:]
    out[:, -1] += rollee[:, 0]

def diffusion_op(u_in, u_out):
  np.copyto(u_out, u_in)
  u_out *= -4
  custom_roll(u_in, +1, 0, u_out)
  custom_roll(u_in, -1, 0, u_out)
  custom_roll(u_in, +1, 1, u_out)
  custom_roll(u_in, -1, 1, u_out)
```

```
Performance counter stats for 'python3 diffusion_custom.py':

   1,112,759,187      cycles                    #    2.898 GHz
   1,475,399,878      instructions              #    1.33  insn per cycle
      72,899,633      cache-references          #  189.860 M/sec
       1,958,765      cache-misses              #    2.687 % of all cache refs
     269,391,116      branches                  #  701.602 M/sec
       3,501,662      branch-misses             #    1.30% of all branches
         383.97 msec task-clock                #    0.998 CPUs utilized
           7,253      faults                    #   18.890 K/sec
           7,253      minor-faults              #   18.890 K/sec
               1      cs                        #    2.604 /sec
               0      migrations                #    0.000 /sec

     0.384685834 seconds time elapsed

     0.360732000 seconds user
     0.024048000 seconds sys
```

cache-misses 11% → 2%
1.21 insn → 1.33 insn
Page-fault 8,800 → 7,200

```
Performance counter stats for 'python3 diffusion_inplace.py':

   1,289,663,038      cycles                    #    2.932 GHz
   1,554,574,423      instructions              #    1.21  insn per cycle
     104,852,495      cache-references          #  238.381 M/sec
      12,237,265      cache-misses              #   11.671 % of all cache refs
     274,143,409      branches                  #  623.261 M/sec
       3,485,377      branch-misses             #    1.27% of all branches
         439.85 msec task-clock                #    0.998 CPUs utilized
           8,807      faults                    #   20.023 K/sec
           8,807      minor-faults              #   20.023 K/sec
               1      cs                        #    2.273 /sec
               0      migrations                #    0.000 /sec

     0.440654486 seconds time elapsed

     0.400665000 seconds user
     0.040066000 seconds sys
```

Key Points

- Reducing the data transfer time to CPU.
- Reducing the amount of work-load that CPU must do.
- Need to deep thinking initialization.
  (한 번에 완료하여 더 이상 계산이 필요없는 연산 포함)
  - Allocating memory
  - Accessing to a file for reading some program configurations
  - Pre-computing parameters (e.g., data mapping function)
- The main pipeline w/o branch is more efficient.
- Data locality, (cache friendly coding)
- Use libraries for fast prototyping

Always having quantitative experiments to see what happening in your system.